Hotel Management System

A DBMS-Based Application for Streamlined Hotel Operations

SK. Azhar Arqham

AP24110011635

SK. Abdul Rasool

AP24110011678

T. Chiranjeevi

AP24110011647

P. Abdul Kalam

AP24110010912

B.Tech Computer Science Engineering — SRM University AP

INTRODUCTION

What is a Hotel Management System?

A **Hotel Management System (HMS)** is a DBMS-backed application designed to digitize and automate the core operations of a hotel — from guest check-in to billing.

🏨 Customer Data

Manage guest profiles and history

📅 Room Booking

Real-time availability and reservations

💳 Billing

Automated invoice and payment tracking

Centralized Data Storage

All guest, room, and transaction data in one unified database

Reduced Manual Work

Replaces paper-based processes with automated workflows

Improved Efficiency

Faster bookings, accurate billing, and real-time room status

OBJECTIVES & MODULES

Goals & System Architecture

Core Objectives

Automate Hotel Operations

Streamline booking, billing, and room allocation end-to-end

Maintain Accurate Records

Eliminate human error with structured database storage

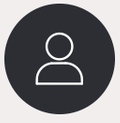
Reduce Data Redundancy

Normalization ensures no duplicate or inconsistent data

Improve Speed & Efficiency

Optimized SQL queries for fast data retrieval and updates

System Modules



Customer Management

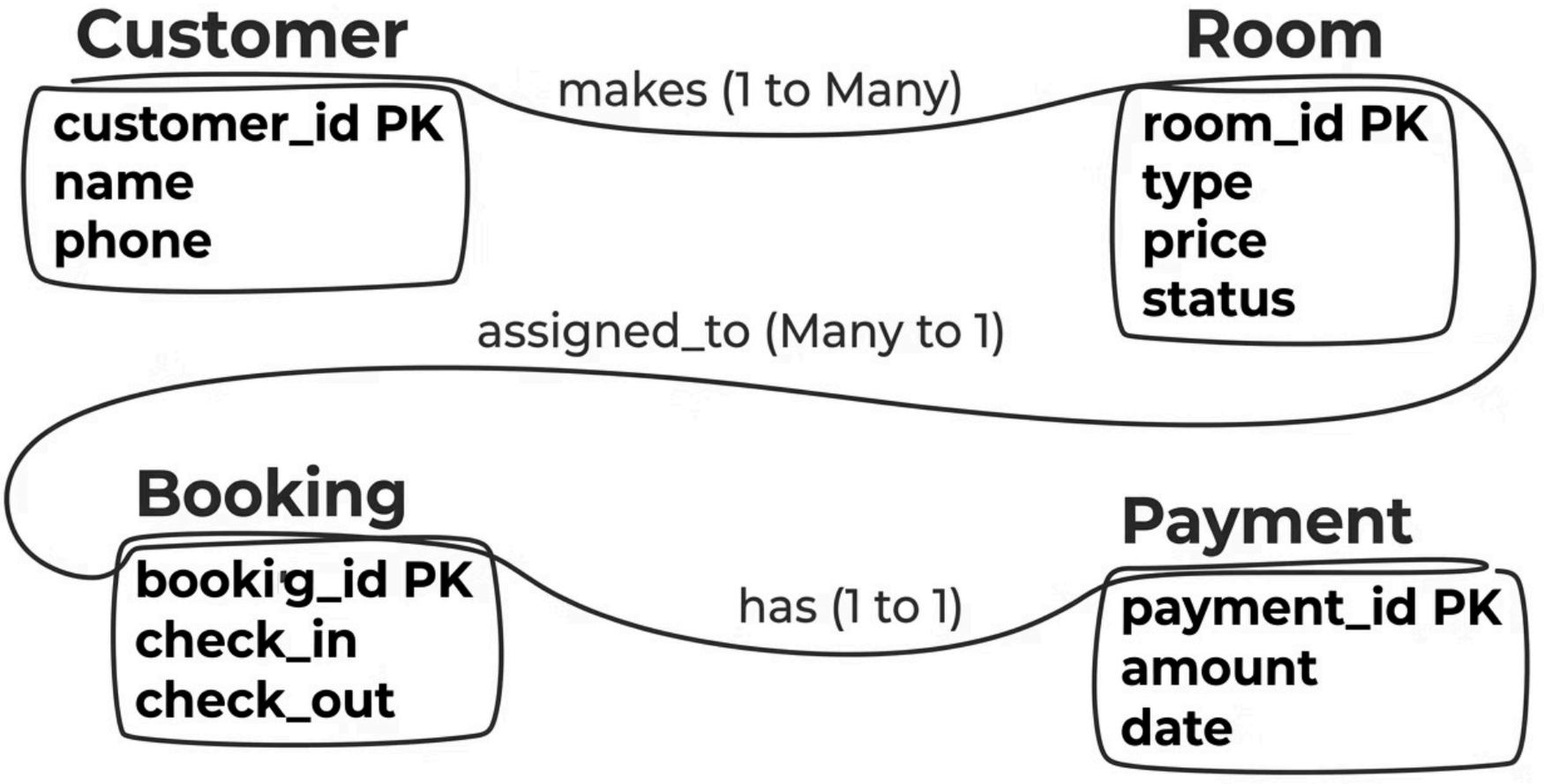
Room Management

Booking System

Payment System

DATABASE DESIGN

Entity-Relationship Diagram



The ER diagram defines four core entities linked through three key relationships — **makes** (1:M), **assigned_to** (M:1), and **has** (1:1) — forming the structural backbone of the HMS database.

DATABASE DESIGN

Database Schema & Tables

Customer

Field	Type
customer_id 🔑	Primary Key
name	VARCHAR
phone	VARCHAR

Room

Field	Type
room_id 🔑	Primary Key
type	VARCHAR
price	DECIMAL
status	ENUM

Booking

Field	Type
booking_id 🔑	Primary Key
customer_id 🔗	Foreign Key
room_id 🔗	Foreign Key
check_in	DATE
check_out	DATE

Payment

Field	Type
payment_id 🔑	Primary Key
booking_id 🔗	Foreign Key
amount	DECIMAL
payment_date	DATE

IMPLEMENTATION

Key SQL Queries

Insert a Customer Record

```
INSERT INTO Customer
VALUES (1, 'Rahul', '9876543210');
```

Select Available Rooms

```
SELECT * FROM Room
WHERE status = 'Available';
```

JOIN: Bookings with Guest & Room Info

```
SELECT Customer.name, Room.room_id
FROM Booking
JOIN Customer
  ON Booking.customer_id =
Customer.customer_id
JOIN Room
  ON Booking.room_id = Room.room_id;
```

Query Categories

1

INSERT

Add new guests, rooms, bookings, and payments to the database

2

SELECT

Retrieve filtered results — e.g., available rooms or active bookings

3

JOIN

Combine data across tables for comprehensive guest reports

NORMALIZATION

Database Normalization Overview

Normalization is the systematic process of structuring a relational database to **minimize redundancy** and **ensure data integrity**. The HMS is normalized up to **Third Normal Form (3NF)**.

1

1NF

Atomic values, no repeating groups

2

2NF

No partial dependency on composite key

3

3NF

No transitive dependency between non-key attributes

NORMALIZATION

Normal Forms: 1NF → 2NF → 3NF

First Normal Form (1NF)

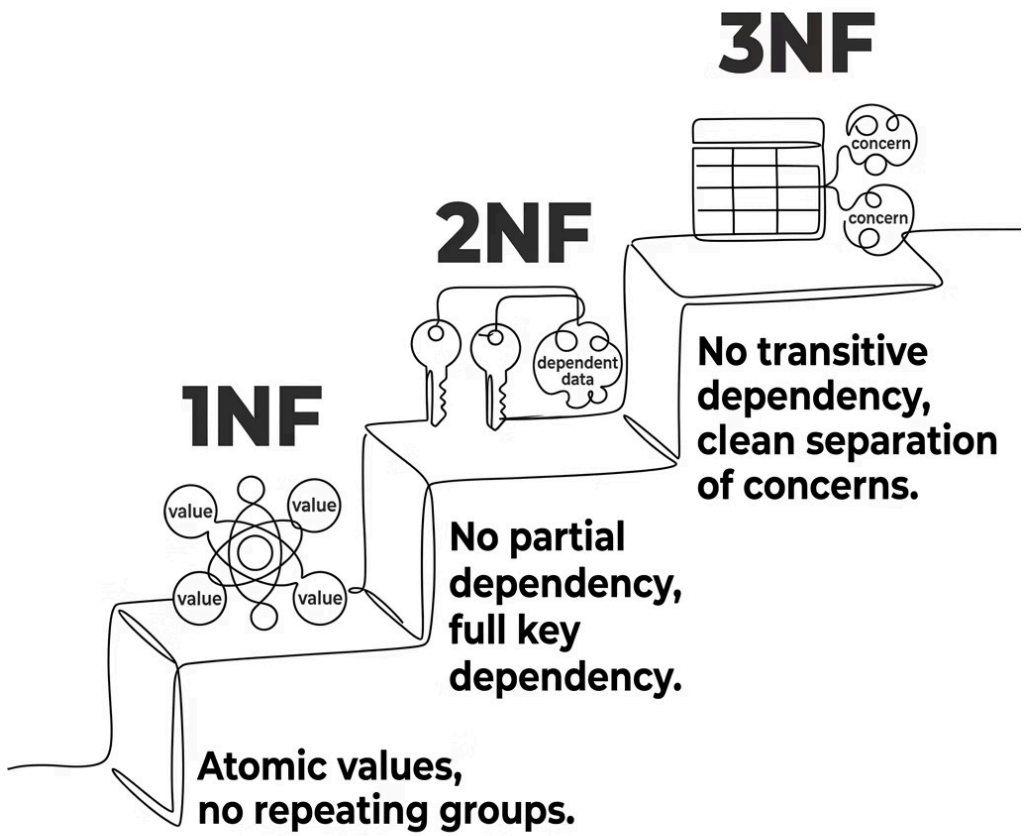
Every attribute must hold **atomic, single values** — no comma-separated room lists or repeating groups.
Before: Customer → Rooms (101,102)
After: Two separate rows — (1|101) and (1|102)

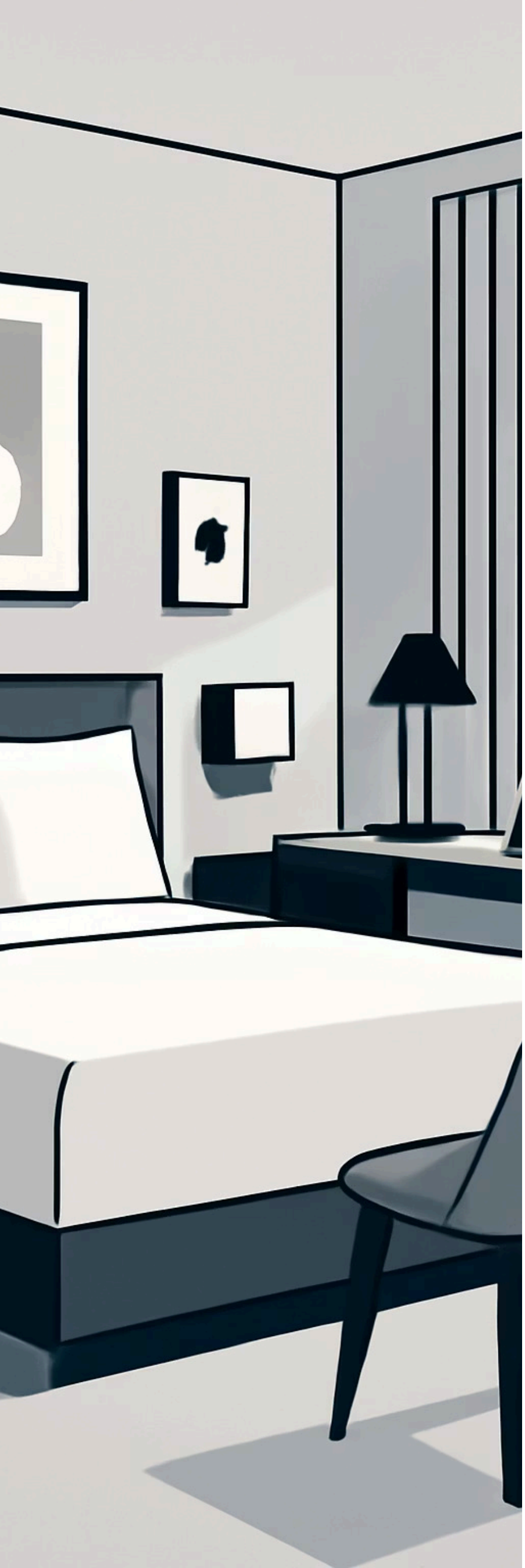
Second Normal Form (2NF)

Satisfies 1NF. All non-key attributes must depend on the **full primary key**, not just part of it.
Customer details (name, phone) are stored in their own table, not duplicated in Booking.

Third Normal Form (3NF)

Satisfies 2NF. Non-key attributes must depend **only on the primary key** — no transitive dependencies.
Room details (type, price) are stored in the Room table, not embedded in Booking.





FEATURES & ADVANTAGES

System Features & Key Advantages

Core Features

Tkinter GUI

User-friendly graphical interface for hotel staff

Optimized SQL

Fast, efficient queries for real-time data access

CRUD Operations

Full Create, Read, Update, Delete support

No Double Booking

Constraint-based logic prevents room conflicts

Advantages

Paperless Operations

Eliminates physical registers and reduces administrative overhead

High Data Accuracy

Enforced constraints ensure consistent, error-free records

Time-Saving

Automated workflows reduce staff effort significantly

FUTURE SCOPE & CONCLUSION

Future Roadmap & Closing Remarks

Future Enhancements

1 Online Booking Portal

Web-based reservations accessible from any browser

2 Mobile Application

Android/iOS app for guests and hotel staff on the go

3 Payment Gateway

Integration with UPI, cards, and net banking platforms

4 Cloud Database

Migrate to AWS/GCP for scalability and remote access

Conclusion

The **Hotel Management System** demonstrates how DBMS principles can be applied to solve real-world operational challenges in the hospitality industry.

- Fully normalized relational database (up to 3NF)
- Modular architecture for easy scalability
- Efficient SQL operations with a clean GUI
- A strong foundation for enterprise-level extension

Built with Python (Tkinter) + SQL — ready to scale.